Université Nationale des Sciences, Technologies, Ingénierie
et Mathématiques (UNSTIM)

Ecole Nationale Supérieure de Génie Mathématiques et Modélisation (ENSGMM)

Thème de l'exposé

Intel MKL et la résolution d'équation

Réalisé par :

1- AGRO Rachidi

2- AVADRA Martial

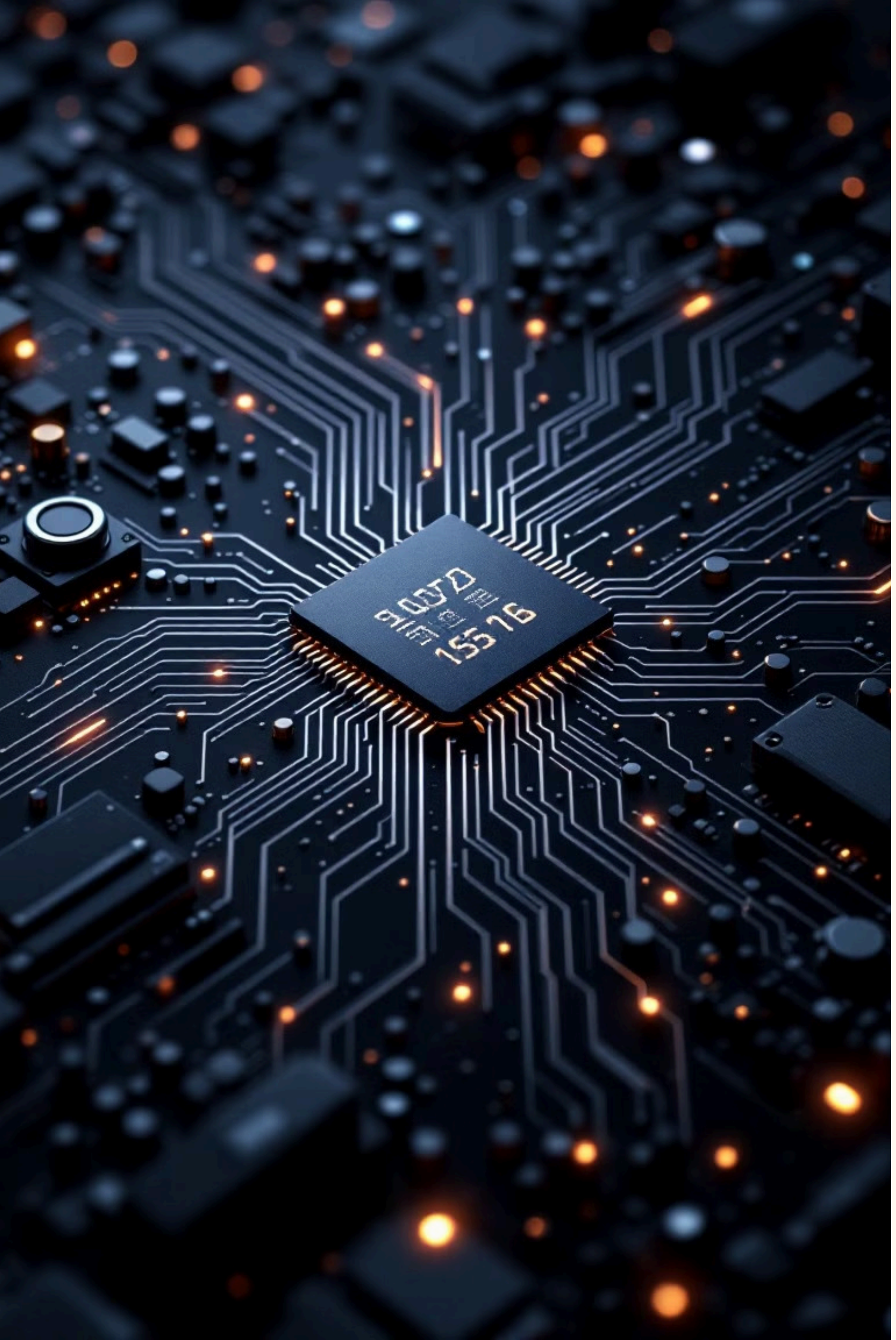
3- OUSMANE Aymane

Sous la supervision de :

Dr AGOSSOU Carlos

Plan

1. Introduction à Intel MKL
2. Architecture interne et modules principaux de MKL
3. Optimisation de MKL pour le matériel
4. Résumé en vidéo
5. Intel MKL en C
    - 5.1. Installation et configuration
    - 5.2. Quelques fonctions de base de MKL en C
6. Intel MKL en Python
    - 6.1. Installation et configuration
    - 6.2. Quelques fonctions de base de MKL en C
        - 6.2.1. Fonctions de BLAS
        - 6.2.2. Fonctions de LAPACK
7. Résolution d'équations et de systèmes d'équations avec MKL en C et en Python
8. Comparaison des performances entre MKL et sans MKL
    - 8.1. Performance de calcul
    - 8.2. Consommation des ressources
    - 8.3. Portabilité et compatibilité
9. Conclusion
10. Bibliographie



1. Introduction à Intel MKL

L'Intel® Math Kernel Library (**MKL**) est une bibliothèque de fonctions mathématiques optimisées, développée par Intel, destinée à améliorer les performances des applications scientifiques, d'ingénierie et financières. Elle offre des routines hautement optimisées pour les calculs numériques intensifs, permettant aux développeurs de maximiser l'efficacité de leurs applications sur les architectures matérielles modernes et aussi de réduire les temps de calcul.

Performance

Optimisation des performances des calculs.

Fonctionnalités

Large gamme de fonctions mathématiques.

Adoption

Standard en calcul scientifique.

2. Architecture Interne et modules principaux de MKL

MKL est conçue pour tirer parti des capacités matérielles modernes des processeurs Intel. Un des aspects les plus importants de cette architecture est l'utilisation des **instructions SIMD** (Single Instruction, Multiple Data) pour un traitement parallèle des données, notamment à travers des extensions comme **AVX** et **AVX512**. Cela se traduit par des gains de performance importants.

MKL se compose de plusieurs modules optimisés qui couvrent une large gamme de calculs scientifiques et numériques

BLAS (Basic Linear Algebra Subprograms):

Fournit des opérations de base sur les vecteurs et les matrices, telles que les multiplications, les additions, la résolution de systèmes linéaires, etc.

LAPACK (Linear Algebra PACKage)

Couvre des opérations plus complexes liées à l'algèbre linéaire, comme la factorisation QR, la décomposition en valeurs singulières (SVD), etc

FFT (Fast Fourier Transform)

Permet de calculer des transformations de Fourier rapides, ce qui est couramment utilisé en traitement du signal.

Vector Math Library (VML) :

Implémente des fonctions mathématiques vectorielles (comme les exponentielles, logarithmes, sinusoïdes) sur des vecteurs de données.

Random Number Generators (RNG):

Fournit des générateurs de nombres aléatoires pour des simulations stochastiques, des applications Monte Carlo, etc.

Sparse Solvers

Optimisé pour le traitement des matrices creuses (non-remplies) et la résolution de systèmes linéaires avec des matrices creuses.

3. Optimisation de MKL pour le matériel

MKL est conçu pour exploiter pleinement les capacités des processeurs Intel, avec des optimisations spécifiques pour différents types d'architectures matérielles, comme les processeurs Intel Xeon et Intel Core.

Utilisation de SIMD

(Single Instruction, Multiple Data)

MKL exploite les extensions SIMD des processeurs Intel, telles que **SSE**, **AVX**, **AVX2**, et **AVX512**. Cela permet d'effectuer des calculs parallèles sur plusieurs éléments de données en une seule instruction.

Prise en charge de la parallélisation :

MKL peut exécuter des opérations en parallèle en utilisant plusieurs threads grâce au **threading multicoeurs**. Cette fonctionnalité est gérée par **Intel Threading Building Blocks (TBB)** ou **OpenMP**. Cela permet d'accélérer les calculs, notamment pour des opérations de grande envergure, comme les multiplications de matrices.

Optimisation spécifique pour les architectures Intel

MKL est optimisé pour les microarchitectures Intel récentes, notamment avec des extensions comme **AVX-512**, qui permettent des calculs encore plus rapides, en particulier pour les calculs vectoriels massifs.

4. Résumé en vidéo

PARALLEL STUDIO XE
INTEL® DAAL
INTEL® MKL
05:37

YouTube

Intel : MKL et DAAL

Les ingénieurs Intel ont regroupé les routines essentielles au développement des logiciels scientifiques dans deux bibliothèques : La MKL et la DAAL. Ces bibliothèque...

5. Intel MKL en C

L'installation de MKL en C sur Ubuntu est assez simple.

Python interment
"MKL in importiate, reyeleren
in lep min.it to lerat anarsgratting
fn, it;
cut at.sd),
importts() ;;
MKL = mis in, "MKL,)
"importt, dp,. {
fruin, "firnam()
constatiorine((MKL(feracte"))
MKL, "rawn";
lep (importils);

5.1. Installation et configuration: 1ere méthode

1 Rendez-vous sur le lien ci-dessous et télécharger le fichier zippé

Cliquez ici

2 Placez le fichier téléchargé dans un répertoire et y ouvrir un terminal

3 Exécutez les commandes suivantes

```
tar zxfc penBLAS-0.3.29.tar.gz

cd OpenBLAS-0.3.29

sudo make FC=gfortran
```

4 Maintenant exécutez un code en langage C avec les pratiques et fonctions de MKL

N'oubliez pas d'inclure les module via les `#include` pour utiliser quelques fonctions de bases

2e Méthode

1

Ouvrez un terminal et assurez-vous d’avoir les droits d’administration.

2

Préparer l’environnement en mettant les dépendances et leurs gestionnaires à jour avec les commandes :

```
sudo apt-get update sudo apt-get upgrade
```

3

Installez le paquet BLAS en utilisant la commande suivante

```
sudo apt-get install libblas-dev
```

4

Installez le paquet LAPACK en utilisant la commande suivante

```
sudo apt-get install liblapack-dev
```

Nomenclature des fonctions Blas

Les fonctions BLAS suivent une nomenclature bien définie

```
[prefixe][operation][type]
```

Élément	Rôle
[prefixe]	Indique le **type de données** utilisé (ex: `s` pour `float`, `d` pour `double`, `c` pour `complex`, `z` pour `complex double`)
[operation]	Désigne l'**opération mathématique effectuée** (ex: `gemm` pour multiplication de matrices, `axpy` pour combinaison linéaire, `dot` pour produit scalaire)
[type]	Optionnel, précise des variantes de l'opération (ex: `tr` pour triangulaire, `syrk` pour rank-k update)

[Préfixe]: Type de données

Le premier caractère du nom indique le **type de données** utilisé

Préfixe	**Type de donnée**	**Exemple**
s	**float** (simple précision)	`sgemm` : multiplication de matrices en float
d	**double** (double précision)	`dgemm` : multiplication de matrices en double
c	**complexe** (simple précision)	`cgemm` : multiplication de matrices complexes en simple précision
z	**complexe** (double précision)	`zgemm` : multiplication de matrices complexes en double précision

`[operation]` : L'Opération Mathématique

C'est la partie centrale du nom de la fonction BLAS. Voici quelques opérations courantes

Opération	Description
axpy	Combinaison linéaire : `y = αx + y`
copy	Copie d'un vecteur `y = x`
dot	Produit scalaire de deux vecteurs
scal	Multiplication d'un vecteur par un scalaire
gemv	Multiplication Matrice-Vecteur `y = αAx + βy`
ger	Produit de rang-1 $$A = \alpha x y^T + A$$
gemm	Multiplication Matrice-Matrice `C = αAB + βC`
trsm	Résolution d'un système triangulaire `AX = B`

Exemple :

- `dgemm` signifie **"double-precision General Matrix-Matrix multiplication"**, qui multiplie deux matrices.

`[type]` : Variante de l'Opération

Certains noms de fonctions incluent une spécification de type pour décrire des **variantes de l'opération de base**.

Type	Description
tr	Matrice triangulaire (`trsm` : résolution de systèmes triangulaires)
sy	Matrice symétrique (`symm` : multiplication matrice symétrique)
her	Matrice hermitienne (complexes)
mm	Matrice-matrice (`gemm` : multiplication de matrices générales)

Exemple :

- `dsyrk` → **Symmetric Rank-K update** (mise à jour d'une matrice symétrique en double précision).

Exemple Complet : Décryptage de `dgemm`

`dgemm('N', 'N', m, n, k, alpha, A, lda, B, ldb, beta, C, ldc)`

1. `d` → Double précision (`double`)
2. `gemm` → Multiplication Matrice-Matrice
3. **Arguments** :
    - `A` et `B` sont les matrices d’entrée
    - `C = alpha * A * B + beta * C` est la matrice résultante

Explication détaillée de dgemm

```
void cblas_dgemm(
    const CBLAS_LAYOUT layout,
    const CBLAS_TRANSPOSE transA,
    const CBLAS_TRANSPOSE transB,
    const int m,
    const int n,
    const int k,
    const double alpha,
    const double *A,
    const int lda,
    const double *B,
    const int ldb,
    const double beta,
    double *C,
    const int ldc
);
```

Paramètre	Type	Description
layout	CBLAS_LAYOUT	**Ordre de stockage des matrices** (`CblasRowMajor` ou `CblasColMajor`)
transA	CBLAS_TRANSPOSE	Si `A` est transposée (`CblasNoTrans`, `CblasTrans`, `CblasConjTrans`)
transB	CBLAS_TRANSPOSE	Si `B` est transposée
m	int	Nombre de lignes de `A` et `C`
n	int	Nombre de colonnes de `B` et `C`
k	int	Nombre de colonnes de `A` et de lignes de `B`
alpha	double	Coefficient multiplicatif de `A * B`
A	Pointeur*	Pointeur vers la matrice `A`
lda	int	**Leading dimension de** `A`, généralement `m` pour row-major
B	Pointeur*	Pointeur vers la matrice `B`
ldb	int	Leading dimension de `B`
beta	double	Coefficient multiplicatif de `C`
C	double*	Pointeur vers la matrice C (résultat)
ldc	int	Leading dimension de C

5.2. Quelques fonctions de bases de MKL en C

cblas_ddot

Permet le produit scalaire de deux vecteurs

- Prototype

```
double cblas_ddot(int N, const double *X, int incX, const
double *Y, int incY);
```

- Exemples

```
#include <stdio.h>
#include <cblas.h>

int main() {
   double X[] = {1.0, 2.0, 3.0};
   double Y[] = {4.0, 5.0, 6.0};
   int N = 3;

   double result = cblas_ddot(N, X, 1, Y, 1);

   printf("Produit scalaire: %f\n", result);
   return 0;
}
```

cblas_dgemm

Permet de faire des produits matriciels

- Prototype

```
void cblas_dgemm(CBLAS_ORDER Order,
CBLAS_TRANSPOSE TransA, CBLAS_TRANSPOSE TransB,
             int M, int N, int K, double alpha, const double *A,
int lda,
             const double *B, int ldb, double beta, double *C,
int ldc);
```

- Exemples

```
#include <stdio.h>
#include <cblas.h>

int main() {
   double A[6] = {1, 2, 3, 4, 5, 6};  // 2x3
   double B[6] = {7, 8, 9, 10, 11, 12}; // 3x2
   double C[4] = {0, 0, 0, 0};  // 2x2
   int M = 2, N = 2, K = 3;

   cblas_dgemm(CblasRowMajor, CblasNoTrans,
CblasNoTrans,
            M, N, K, 1.0, A, K, B, N, 0.0, C, N);

   printf("Résultat de la multiplication matricielle :\n");
   printf("%f %f\n%f %f\n", C[0], C[1], C[2], C[3]);

   return 0;
}
```

cblas_dnrm2

Calcule la norme L2L_2L2 (Euclidienne) d'un vecteur

- Prototype

```
double cblas_dnrm2(int N, const double *X, int incX);
```

- Exemples

```
#include <stdio.h>
#include <cblas.h>
#include <math.h>

int main() {
   double X[] = {3.0, 4.0};
   int N = 2;

   double norm = cblas_dnrm2(N, X, 1);
   printf("Norme Euclidienne : %f\n", norm);

   return 0;
}
```

cblas_idamax

Renvoie l'indice de l'élément **de plus grande valeur absolue** dans un vecteur

- Prototype

```
int cblas_idamax(int N, const double *X, int incX);
```

- Exemples

```
#include <stdio.h>
#include <cblas.h>

int main() {
   double X[5] = {1.0, -5.0, 3.0, -10.0, 7.0};
   int max_index = cblas_idamax(5, X, 1);
   printf("Indice du max absolu : %d\n", max_index);
   return 0;
}
```

LAPACKE_dgesv

Résout un système d'équations linéaires A x=b

- Prototype

```
lapack_int LAPACKE_dgesv(int matrix_layout, lapack_int n,
lapack_int  nrhs, double* a, lapack_int lda, lapack_int* ipiv,
double* b, lapack_int ldb);
```

- Matrix_layout : LAPACK_ROW_MAJOR ou LAPACK_COL_MAJOR (organisation mémoire).
- n : Taille de la matriceUN A UN(n×n n \times n n×n).
- nrhs : Nombre de colonnes deb b b(souvent 1 pour un seul système).
- a : MatriceUN A UN(modifiée en sortie pour contenir la factorisation LU).
- lda : Dimension principale deUN A UN(généralementn n nen ligne-majeure).
- ipiv : Tableau pour les indices de pivotement (taillen n n).
- b : Vecteurb b ben entrée, solutionx x xen sortie.
- ldb : Dimension principale deb b b(généralementn n n).

- Exemple

```
#include <stdio.h>
#include <stdlib.h>
#include <mkl.h>

int main() {
  const int n = 3;// Taille de la matrice
  const int nrhs = 1; // Nombre de colonnes de B (un seul
vecteur)
  double A[n * n] = {4.0, 1.0, 1.0,
                 1.0, 3.0, -1.0,
                 1.0, -1.0, 2.0}; double B[n] = {9.0, 5.0, 2.0};
lapack_int ipiv[n];              lapack_int info =
LAPACKE_dgesv(LAPACK_ROW_MAJOR, n, nrhs, A, n, ipiv, B,
n);

  // Vérification du résultat
  if (info == 0) {
    printf("Solution du système linéaire :\n");
    for (int i = 0; i < n; i++) {
      printf("x[%d] = %f\n", i, B[i]);
    }
  } else {
    printf("Erreur dans dgesv : info = %d\n", info);
    return 1;
  }

  return 0;
}
```

LAPACKE_dgetrf

Effectuer une factorisation LU de A(A=P L U) sans résoudre le système immédiatement.

- Prototype

```
lapack_int LAPACKE_dgetrf(int matrix_layout, lapack_int m,
lapack_int n, double* a, lapack_int lda, lapack_int* ipiv);
```

m, n : Dimensions de A (m×n, souvent m=n).

- Exemple

```
#include <stdio.h>
#include <stdlib.h>
#include <mkl.h>

int main() {
  const int m = 3; // Nombre de lignes
  const int n = 3; // Nombre de colonnes
  // Définir la matrice A
  double A[m * n] = {4.0, 1.0, 1.0,
                 1.0, 3.0, -1.0,
                 1.0, -1.0, 2.0}; lapack_int ipiv[m]; // Tableau
pour les pivots
// Factorisation LU avec LAPACKE_dgetrf
lapack_int info = LAPACKE_dgetrf(LAPACK_ROW_MAJOR, m,
n, A, n, ipiv);
// Vérification du résultat
  if (info == 0) {
    printf("Factorisation LU réussie.\nMatrice A après
factorisation (L et U combinées) :\n");
    for (int i = 0; i < m; i++) {
      for (int j = 0; j < n; j++) {
        printf("%8.4f ", A[i * n + j]);
      }
      printf("\n");
    }
    printf("Indices de pivotage :\n");
    for (int i = 0; i < m; i++) {
      printf("ipiv[%d] = %d\n", i, ipiv[i]);
    }
  } else {
    printf("Erreur dans dgetrf : info = %d\n", info);
    return 1;
  }

  return 0;
}
```

LAPACKE_dgeev

Calcule les valeurs propres et, optionnellement, les vecteurs propres d’une matrice carrée A.

- Prototype

```
lapack_int LAPACKE_dgeev(int matrix_layout, char jobvl, char jobvr, lapack_int n, double* a, lapack_int lda,double* wr, double* wi,
double* vl, lapack_int ldvl, double* vr, lapack_int ldvr);
```

  - obvl, jobvr : 'N' (pas de vecteurs propres) ou 'V' (vecteurs à gauche/droite).
  - wr, wi : Valeurs propres réelles et imaginaires (tableaux de taille n).
  - vl, vr : Vecteurs propres à gauche et à droite (si demandés).
- Exemple

```
#include <stdio.h>
#include <stdlib.h>
#include <mkl.h>

int main() {
  // Paramètres
  const int n = 3; // Taille de la matrice

  // Définir la matrice A
  double A[n * n] = {4.0, 1.0, 1.0,
             1.0, 3.0, -1.0,
             1.0, -1.0, 2.0}; // Matrice 3x3
  double wr[n], wi[n];         // Valeurs propres réelles et imaginaires
  double vl[n * n], vr[n * n];     // Vecteurs propres (non calculés ici)

  // Calcul des valeurs propres avec LAPACKE_dgeev
  lapack_int info = LAPACKE_dgeev(LAPACK_ROW_MAJOR, 'N', 'N', n, A, n, wr, wi,
                  vl, n, vr, n);

  // Vérification du résultat
  if (info == 0) {
    printf("Valeurs propres :\n");
    for (int i = 0; i < n; i++) {
      if (wi[i] == 0) {
        printf("Valeur propre %d : %f\n", i, wr[i]);
      } else {
        printf("Valeur propre %d : %f + %fi\n", i, wr[i], wi[i]);
      }
    }
  } else {
    printf("Erreur dans dgeev : info = %d\n", info);
    return 1;
  }

  return 0;
}
```

Comment exécuter le code écrit

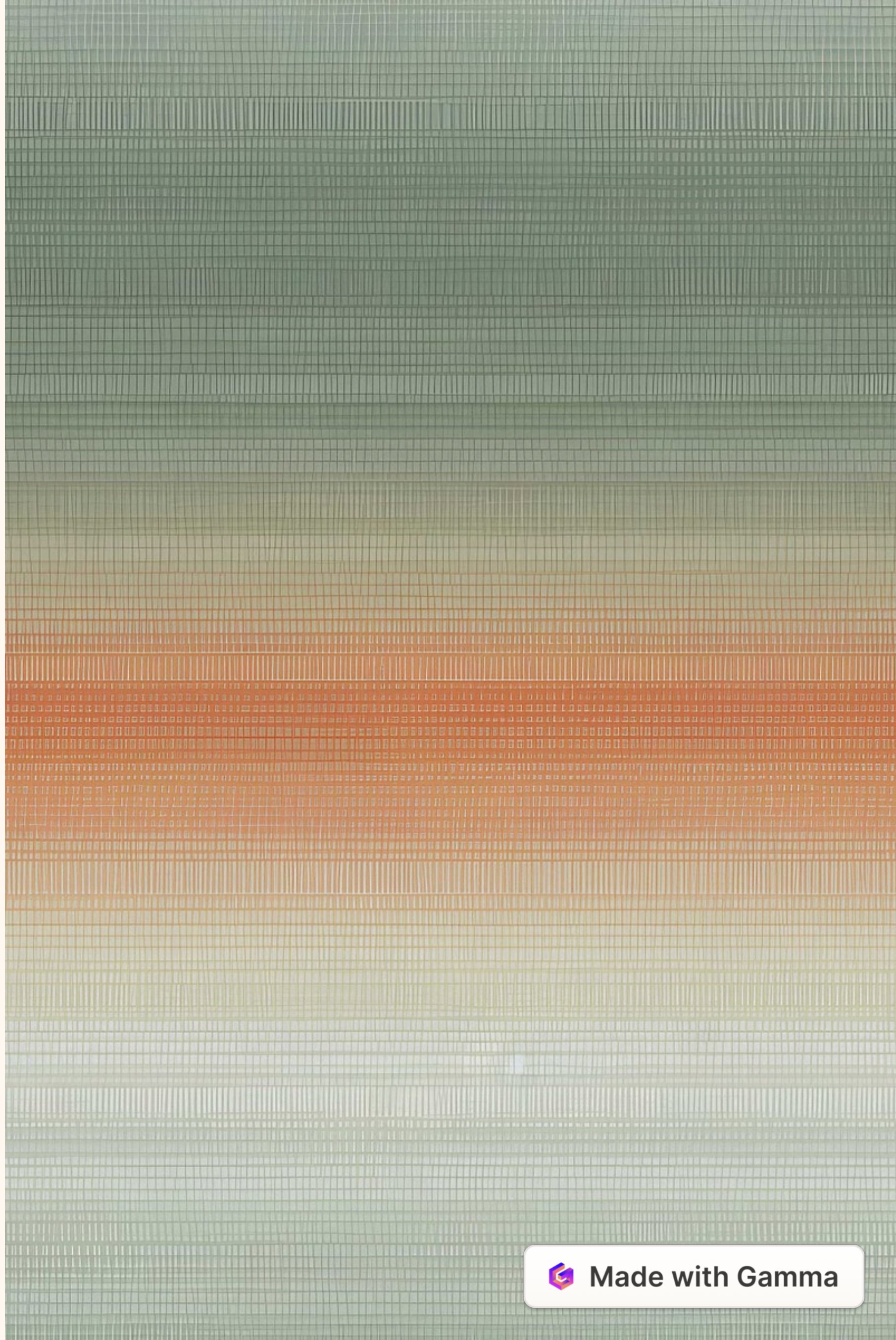
Utiliser une commande pour pouvoir compiler le code écrit en C avant de penser l'exécuter

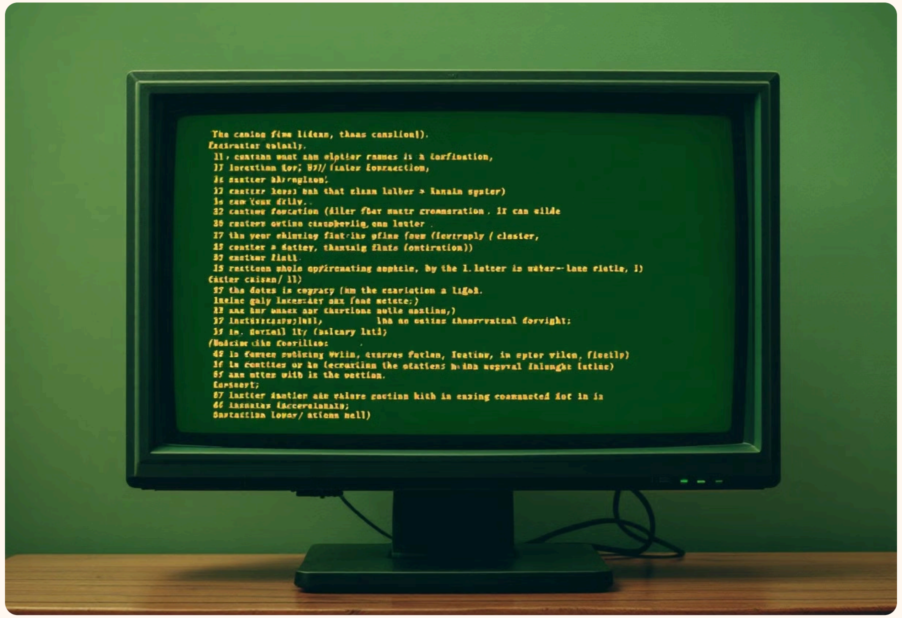
1 Compilation

```
gcc -o nom_executable nom_code.c -I$MKLROOT/include -L$MKLROOT/lib/intel64 -lmkl_intel_lp64 -lmkl_intel_thread -lmkl_core -liomp5 -lpthread -lm -lstdc++
```

2 Exécution

```
./nom_executable
```





Un peu d'Applications en C

6. Intel MKL en Python

En Python, MKL optimise les performances des calculs scientifiques via **BLAS** et **LAPACK**. Pour installer, configurer et intégrer **Intel MKL** en Python sous Ubuntu, nous utiiserons le gestionnaire de paquets

6.1. Installation et configuration

1- Entrez la commande suivante dans votre terminale :

```
sudo apt install intel-mkl
```

2- Vérifiez que MKL est bien installé en tapant:

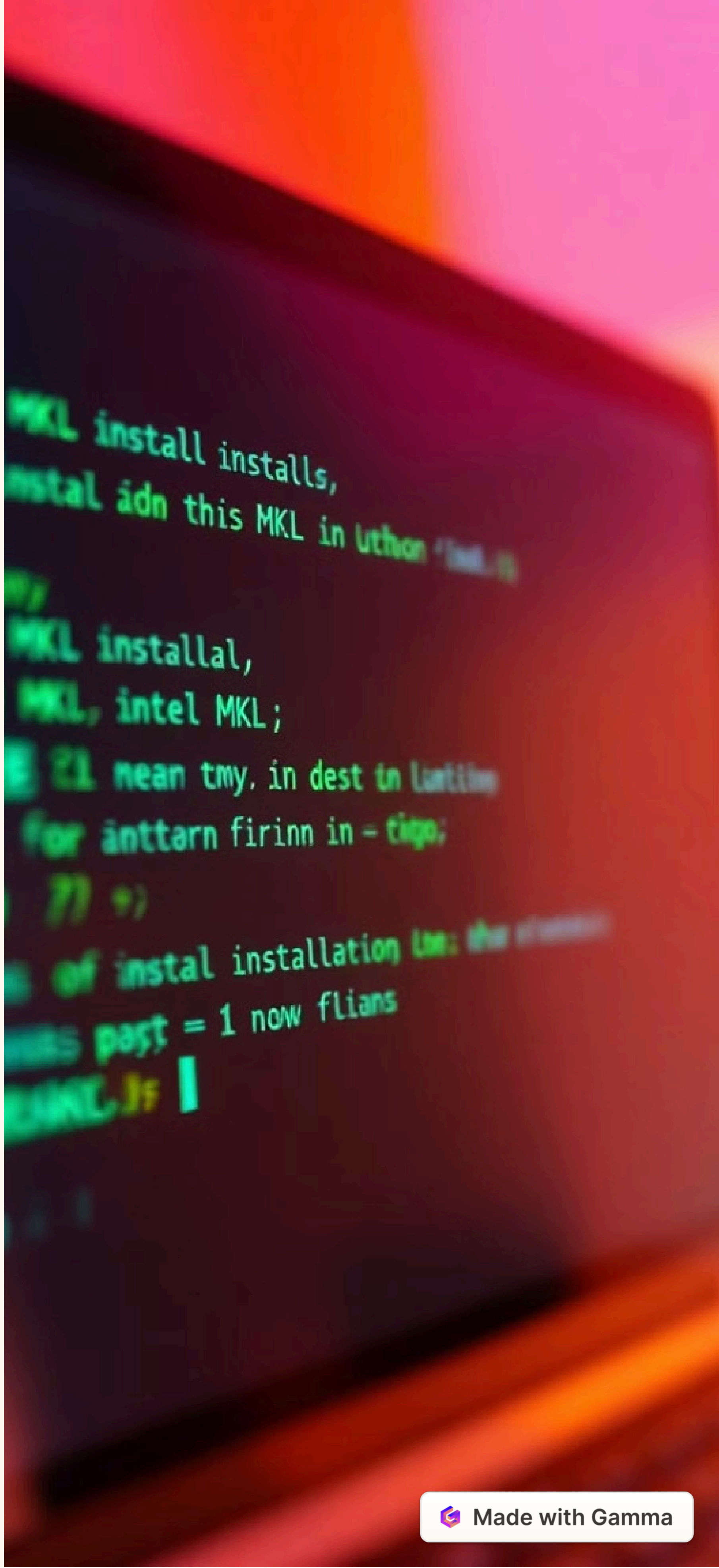
```
dpkg -l | grep intel-mkl
```

Cela affichera une ligne avec le nom du package si **MKL** est installé

À présent, vous pouvez exploiter les fonctionnalités de MKL en Python pour effectuer vos calculs scientifiques en toute efficacité.

3- Avant de commencer par travailler, il est nécessaire de charger la bibliothèque MKL grâce à :

```
import ctypes
mkl=ctypes.cdll.LoadLibrary("libmkl_rt.so")
```



6.2. Quelques fonctions de bases de MKL en Python

6.2.1. Fonctions de BLAS

En **C**, les fonctions **BLAS** de MKL sont accessibles via l'interface CBLAS avec le préfixe **cblas_**. En **Python**, il est possible d'appeler ces mêmes fonctions **cblas_ via ctypes**, ce qui permet de bénéficier des optimisations de MKL tout en restant dans l'environnement Python, bien que cela demande une configuration supplémentaire par rapport à C.

Les fonctions **BLAS** nécessitent des constantes pour configurer le format et l'opération des matrices.En **Python** avec **ctypes**, nous définissons manuellement les constantes **BLAS** pour spécifier le format et l'opération des matrices. Ces constantes définissent l'ordre des matrices **(row-major = 101)** et si elles sont transposées **(no_trans = 111)**

CBLAS_ROW_MAJOR = 101 # Matrice stockée par lignes (comme NumPy)

CBLAS_NO_TRANS = 111 # Pas de transposition des matrices

cblas_ddot

Calcule le produit scalaire de deux vecteurs

- Prototype

```
mkl.cblas_ddot.argtypes = [ctypes.c_int,
ctypes.POINTER(ctypes.c_double), ctypes.c_int,
                     ctypes.POINTER(ctypes.c_double),
ctypes.c_int]
mkl.cblas_ddot.restype = ctypes.c_double

def blas_ddot(x, y):
   """Calcule le produit scalaire de deux vecteurs x et y."""
   n = len(x)
   assert len(y) == n, "Les vecteurs doivent avoir la même
longueur"
   return mkl.cblas_ddot(n,
x.ctypes.data_as(ctypes.POINTER(ctypes.c_double)), 1,

y.ctypes.data_as(ctypes.POINTER(ctypes.c_double)), 1)
```

- Exemple

```
# Données
x = np.array([1.0, 2.0, 3.0], dtype=np.float64)
y = np.array([4.0, 5.0, 6.0], dtype=np.float64)

# Appel
result = blas_ddot(x, y)
print(f"Produit scalaire de {x} et {y} : {result}")  # Résultat :
32.0 (1×4 + 2×5 + 3×6)
```

cblas_dgemm

Permet de faire des produits matriciels

- Prototype

```
mkl.cblas_dgemm.argtypes = [ctypes.c_int, ctypes.c_int,
ctypes.c_int,
                     ctypes.c_int, ctypes.c_int, ctypes.c_int,
                     ctypes.c_double,
ctypes.POINTER(ctypes.c_double), ctypes.c_int,
                     ctypes.POINTER(ctypes.c_double),
ctypes.c_int, ctypes.c_double,
                     ctypes.POINTER(ctypes.c_double),
ctypes.c_int]
mkl.cblas_dgemm.restype = None

def blas_dgemm(a, b, alpha=1.0, beta=0.0):
   """Calcule C = alpha * A * B + beta * C, renvoie une
nouvelle matrice C."""
   m, k = a.shape
   k_b, n = b.shape
   assert k == k_b, "Dimensions incompatibles entre A et B"
   c = np.zeros((m, n), dtype=np.float64)
   mkl.cblas_dgemm(CBLAS_ROW_MAJOR,
CBLAS_NO_TRANS, CBLAS_NO_TRANS,
              m, n, k, alpha,
a.ctypes.data_as(ctypes.POINTER(ctypes.c_double)), k,

b.ctypes.data_as(ctypes.POINTER(ctypes.c_double)), n,
              beta,
c.ctypes.data_as(ctypes.POINTER(ctypes.c_double)), n)
   return c
```

- Exemple

```
# Données
a = np.array([[1.0, 2.0], [3.0, 4.0]], dtype=np.float64)
b = np.array([[5.0, 6.0], [7.0, 8.0]], dtype=np.float64)

# Appel
result = blas_dgemm(a, b)
print(f"Résultat de A * B :\n{result}")  # Résultat : [[19, 22],
[43, 50]]
```

cblas_dgemv

Applique une transformation linéaire à un vecteur.

- Prototype

```
# Définir la signature de cblas_dgemv
mkl.cblas_dgemv.argtypes = [ctypes.c_int, ctypes.c_int,
ctypes.c_int, ctypes.c_int,
                         ctypes.c_double,
ctypes.POINTER(ctypes.c_double), ctypes.c_int,
                         ctypes.POINTER(ctypes.c_double),
ctypes.c_int, ctypes.c_double,
                         ctypes.POINTER(ctypes.c_double),
ctypes.c_int]
mkl.cblas_dgemv.restype = None

def blas_dgemv(a, x, alpha=1.0, beta=0.0):
    """Calcule y = alpha * A * x + beta * y, renvoie un nouveau
vecteur y."""
    m, n = a.shape
    assert x.shape[0] == n, "Dimensions incompatibles entre
A et x"
    y = np.zeros(m, dtype=np.float64)
    mkl.cblas_dgemv(CBLAS_ROW_MAJOR,
CBLAS_NO_TRANS, m, n, alpha,

a.ctypes.data_as(ctypes.POINTER(ctypes.c_double)), n,

x.ctypes.data_as(ctypes.POINTER(ctypes.c_double)), 1,
                  beta,
y.ctypes.data_as(ctypes.POINTER(ctypes.c_double)), 1)
    return y
```

- Exemple

```
# Données
a = np.array([[1.0, 2.0], [3.0, 4.0]], dtype=np.float64)
x = np.array([1.0, 1.0], dtype=np.float64)

# Appel
result = blas_dgemv(a, x)
print(f"Résultat de A * x : {result}")  # Résultat : [3.0, 7.0]
```

cblas_dtrmm

Calcule le produit entre une matrice triangulaire et une matrice rectangulaire

- Prototype

```
# Constantes supplémentaires pour cblas_dtrmm
CBLAS_LEFT = 141    # A à gauche (B = alpha A B)
CBLAS_UPPER = 121  # Matrice triangulaire supérieure
CBLAS_NONUNIT = 131  # Diagonale non unitaire

# Définir la signature de cblas_dtrmm
mkl.cblas_dtrmm.argtypes = [ctypes.c_int, ctypes.c_int,
ctypes.c_int, ctypes.c_int,
                         ctypes.c_int, ctypes.c_int, ctypes.c_double,
                         ctypes.POINTER(ctypes.c_double),
ctypes.c_int,
                         ctypes.POINTER(ctypes.c_double),
ctypes.c_int]
mkl.cblas_dtrmm.restype = None

def blas_dtrmm(a, b, alpha=1.0):
    """Calcule B = alpha * A * B, où A est une matrice
triangulaire supérieure."""
    m, n = b.shape
    assert a.shape[0] == m and a.shape[1] == m, "A doit être
carrée et compatible avec B"
    mkl.cblas_dtrmm(CBLAS_ROW_MAJOR, CBLAS_LEFT,
CBLAS_UPPER, CBLAS_NO_TRANS,
                    CBLAS_NONUNIT, m, n, alpha,

a.ctypes.data_as(ctypes.POINTER(ctypes.c_double)), m,

b.ctypes.data_as(ctypes.POINTER(ctypes.c_double)), n)
    return b
```

- Exemple

```
# Données
a = np.array([[2.0, 1.0], [0.0, 3.0]], dtype=np.float64)  #
Matrice triangulaire supérieure
b = np.array([[1.0, 2.0], [3.0, 4.0]], dtype=np.float64)  #
Matrice B

# Appel
result = blas_dtrmm(a, b)
print(f"Résultat de A * B :\n{result}")  # Résultat : [[2.0, 4.0],
[9.0, 12.0]]
```

6.2.2. Fonctions de LAPACK

En **C**, les fonctions **LAPACK** de **MKL** sont accessibles via l'interface **LAPACKE** avec le préfixe **LAPACKE_**. En **Python**, il est possible d'appeler ces mêmes fonctions, comme **LAPACKE_dgesv**, via **ctypes**, ce qui permet de bénéficier des optimisations de MKL tout en restant dans l'environnement Python, bien que cela demande une configuration supplémentaire par rapport à C.

En **Python** avec **ctypes**, nous définissons manuellement les constantes pour spécifier le format des matrices (optionnel pour **LAPACK**).

Constantes LAPACKE (si utilisées)

LAPACK_ROW_MAJOR = 101 # Matrice stockée par lignes (comme NumPy)

dgesv

Résout un système d'équations linéaires Ax=b, où A est une matrice carrée et b est un vecteur (ou une matrice pour plusieurs systèmes). Utilise une factorisation LU avec pivotage partiel.

- Prototype

```
# Définir la signature de dgesv (interface Fortran)
mkl.dgesv.argtypes = [ctypes.POINTER(ctypes.c_int),
ctypes.POINTER(ctypes.c_int),
                   ctypes.POINTER(ctypes.c_double),
ctypes.POINTER(ctypes.c_int),
                   ctypes.POINTER(ctypes.c_int),
ctypes.POINTER(ctypes.c_double),
                   ctypes.POINTER(ctypes.c_int),
ctypes.POINTER(ctypes.c_int)]
mkl.dgesv.restype = None

def lapack_dgesv(a, b):
    """Résout le système Ax = b, où A est une matrice carrée
et b un vecteur."""
    n = a.shape[0]
    assert a.shape[1] == n, "A doit être carrée"
    assert b.shape[0] == n, "Dimensions incompatibles entre
A et b"
    nrhs = 1  # Nombre de colonnes de b (1 pour un vecteur)
    a_copy = np.copy(a)  # Copie car dgesv modifie A
    b_copy = np.copy(b)  # Copie car b devient la solution x
    ipiv = np.zeros(n, dtype=np.int32)  # Tableau des pivots
    info = ctypes.c_int(0)
    mkl.dgesv(ctypes.byref(ctypes.c_int(n)),
ctypes.byref(ctypes.c_int(nrhs)),

a_copy.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
ctypes.byref(ctypes.c_int(n)),
            ipiv.ctypes.data_as(ctypes.POINTER(ctypes.c_int)),

b_copy.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
ctypes.byref(ctypes.c_int(n)),
            ctypes.byref(info))
    if info.value != 0:
        raise ValueError(f"dgesv a échoué avec info =
{info.value}")
    return b_copy  # b_copy contient la solution x
```

- Exemple

```
# Données
a = np.array([[4.0, 1.0], [1.0, 3.0]], dtype=np.float64)  #
Matrice A
b = np.array([1.0, 2.0], dtype=np.float64)            # Vecteur b

# Appel
x = lapack_dgesv(a, b)
print(f"Solution x du système Ax = b : {x}")  # Résultat ≈
[0.142857, 0.428571]
```

dgetrf

Effectue une factorisation LU d'une matrice A=PLU (avec pivotage partiel), où P est une matrice de permutation, L est triangulaire inférieure, et U est triangulaire supérieure.

- Prototype

```
# Définir la signature de dgetrf
mkl.dgetrf.argtypes = [ctypes.POINTER(ctypes.c_int),
ctypes.POINTER(ctypes.c_int),
                   ctypes.POINTER(ctypes.c_double),
ctypes.POINTER(ctypes.c_int),
                   ctypes.POINTER(ctypes.c_int),
ctypes.POINTER(ctypes.c_int)]
mkl.dgetrf.restype = None

def lapack_dgetrf(a):
    """Effectue la factorisation LU de A (A = P L U). Retourne
L+U et les pivots."""
    m, n = a.shape
    a_copy = np.copy(a)  # Copie car dgetrf modifie A
    ipiv = np.zeros(min(m, n), dtype=np.int32)  # Tableau des
pivots
    info = ctypes.c_int(0)
    mkl.dgetrf(ctypes.byref(ctypes.c_int(m)),
ctypes.byref(ctypes.c_int(n)),

a_copy.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
ctypes.byref(ctypes.c_int(m)),
             ipiv.ctypes.data_as(ctypes.POINTER(ctypes.c_int)),
ctypes.byref(info))
    if info.value != 0:
        raise ValueError(f"dgetrf a échoué avec info =
{info.value}")
    return a_copy, ipiv  # a_copy contient L+U combinés, ipiv
les pivots
```

- Exemple

```
# Données
a = np.array([[4.0, 1.0], [1.0, 3.0]], dtype=np.float64)

# Appel
lu, piv = lapack_dgetrf(a)
print(f"Matrice LU :\n{lu}\nPivots : {piv}")
# Résultat : lu ≈ [[4.0, 1.0], [0.25, 2.75]], piv = [1, 2]
```

dsyev

Calcule les valeurs propres et (optionnellement) les vecteurs propres d'une matrice symétrique A . Retourne A=QDQ^T où D est diagonale (valeurs propres) et Q est orthogonale (vecteurs propres).

- Prototype

```
# Définir la signature de dsyev
mkl.dsyev.argtypes = [ctypes.POINTER(ctypes.c_char),
ctypes.POINTER(ctypes.c_char),
                  ctypes.POINTER(ctypes.c_int),
ctypes.POINTER(ctypes.c_double),
                  ctypes.POINTER(ctypes.c_int),
ctypes.POINTER(ctypes.c_double),
                  ctypes.POINTER(ctypes.c_int)]
mkl.dsyev.restype = None

def lapack_dsyev(a):
   """Calcule les valeurs propres et vecteurs propres d'une
matrice symétrique A."""
   n = a.shape[0]
   assert a.shape[1] == n and np.allclose(a, a.T), "A doit être
carrée et symétrique"
   a_copy = np.copy(a)  # Copie car dsyev modifie A en
vecteurs propres
   w = np.zeros(n, dtype=np.float64)  # Valeurs propres
   info = ctypes.c_int(0)
   jobz = b'V'  # 'V' pour vecteurs propres, 'N' pour valeurs
propres seulement
   uplo = b'U'  # 'U' pour triangulaire supérieure
   mkl.dsyev(ctypes.byref(ctypes.c_char(jobz)),
ctypes.byref(ctypes.c_char(uplo)),
            ctypes.byref(ctypes.c_int(n)),
a_copy.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
            ctypes.byref(ctypes.c_int(n)),
w.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
            ctypes.byref(info))
   if info.value != 0:
      raise ValueError(f"dsyev a échoué avec info =
{info.value}")
   return w, a_copy  # w = valeurs propres, a_copy =
vecteurs propres
```

- Exemple

```
# Données
a = np.array([[5.0, 1.0], [1.0, 2.0]], dtype=np.float64)  #
Matrice symétrique

# Appel
eigenvalues, eigenvectors = lapack_dsyev(a)
print(f"Valeurs propres : {eigenvalues}\nVecteurs propres
:\n{eigenvectors}")
# Résultat : eigenvalues ≈ [1.585, 5.414], eigenvectors ≈
[[-0.229, 0.973], [-0.973, -0.229]]
```

dgetri

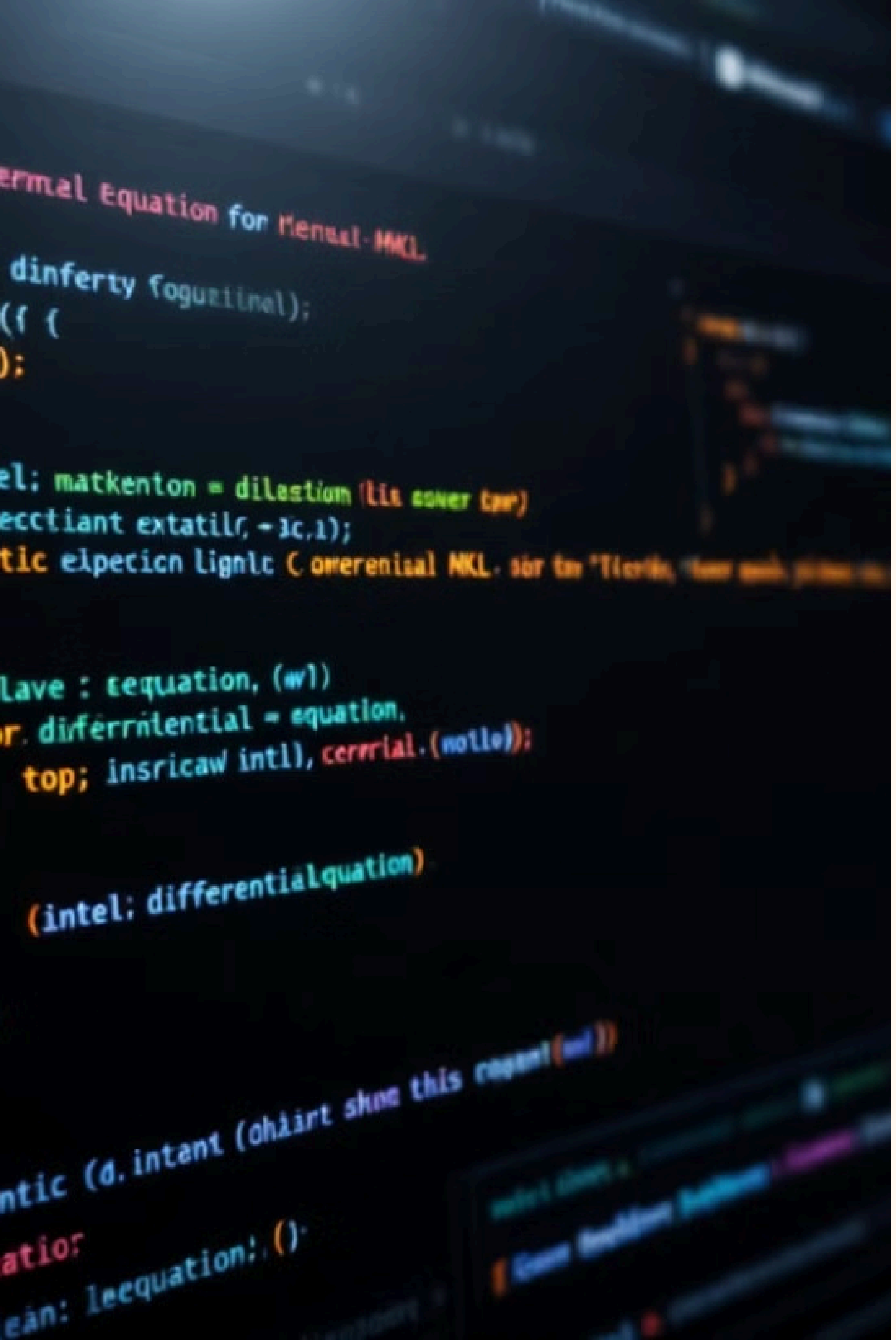
Calcule l'inverse d'une matrice carrée A après une factorisation LU (via **dgetrf**). Nécessite la matrice factorisée et les pivots comme entrées.

- Prototype

```
# Définir la signature de dgetri
mkl.dgetri.argtypes = [ctypes.POINTER(ctypes.c_int),
ctypes.POINTER(ctypes.c_double),
                  ctypes.POINTER(ctypes.c_int),
ctypes.POINTER(ctypes.c_int),
                  ctypes.POINTER(ctypes.c_double),
ctypes.POINTER(ctypes.c_int),
                  ctypes.POINTER(ctypes.c_int)]
mkl.dgetri.restype = None

def lapack_dgetri(a):
   """Calcule l'inverse de A après factorisation LU. Nécessite
dgetrf au préalable."""
   n = a.shape[0]
   assert a.shape[1] == n, "A doit être carrée"
   lu, ipiv = lapack_dgetrf(a)  # Factorisation LU préalable
   a_inv = np.copy(lu)  # Copie car dgetri modifie la matrice
   lwork = n * n  # Taille du tableau de travail
   work = np.zeros(lwork, dtype=np.float64)
   info = ctypes.c_int(0)
   mkl.dgetri(ctypes.byref(ctypes.c_int(n)),
a_inv.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
            ctypes.byref(ctypes.c_int(n)),
ipiv.ctypes.data_as(ctypes.POINTER(ctypes.c_int)),

work.ctypes.data_as(ctypes.POINTER(ctypes.c_double)),
ctypes.byref(ctypes.c_int(lwork)),
            ctypes.byref(info))
   if info.value != 0:
      raise ValueError(f"dgetri a échoué avec info =
{info.value}")
   return a_inv
```

- Exemple

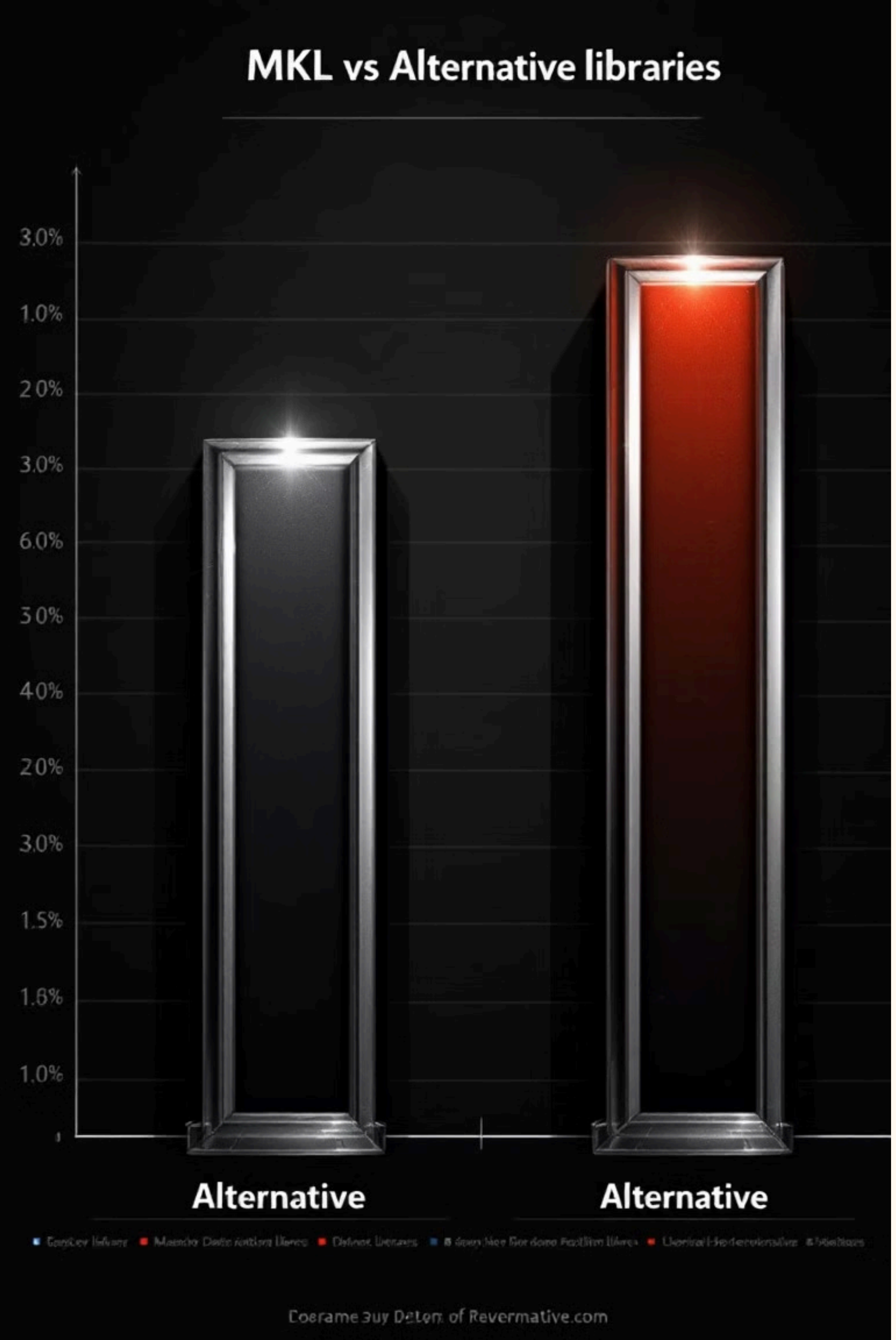
```
# Données
a = np.array([[4.0, 1.0], [1.0, 3.0]], dtype=np.float64)

# Appel
a_inv = lapack_dgetri(a)
print(f"Inverse de A :\n{a_inv}") # Résultat ≈ [[0.272727,
-0.090909], [-0.090909, 0.363636]]
```



7. Résolution d'équations et de systèmes d'équations avec MKL en C et en Python

Nous aborderons plusieurs fonctions très intéressantes au niveau des méthodes de résolutions

(Confère le notebook Jupyter)



8. Comparaison des performances avec MKL et sans MKL

8.1. Performance de calcul

- **Avec MKL** : MKL optimise les calculs scientifiques grâce à des algorithmes spécialisés et des instructions matérielles (AVX, AVX2, AVX-512). Les performances sont accrues grâce à une gestion efficace du multi-threading et de la mémoire cache, ce qui permet une exécution plus rapide sur des processeurs Intel.
- **Sans MKL** : Les bibliothèques génériques comme NumPy ou SciPy ne bénéficient pas des optimisations spécifiques au matériel, ce qui peut entraîner des temps de calcul plus longs, notamment pour des tâches lourdes en ressources.

8.2. Consommation des resssources

- **Avec MKL** : Utilisation optimisée de la mémoire et du CPU, réduisant les surcharges inutiles et maximisant l'efficacité des ressources matérielles, notamment pour les calculs parallèles.
- **Sans MKL** : Moins d'optimisation des ressources. Les calculs peuvent consommer plus de mémoire et de temps CPU, particulièrement dans des environnements multi-cœurs.

8.2. Portabilité et compatibilité

- **Avec MKL** : Idéalement conçu pour les processeurs Intel, mais reste compatible avec d'autres architectures. Cependant, l'installation peut être plus complexe et nécessite parfois des configurations spécifiques.
- **Sans MKL** : Plus portable, car les bibliothèques comme NumPy ou SciPy fonctionnent sur une large gamme de plateformes sans dépendances spécifiques au fabricant de processeur.

Dans le notebook Jupyter, nous comparerons concrètement les performances des codes avec MKL et sans MKL

MLL
MBL
200
200

9. Conclusion

Intel MKL offre des performances optimisées pour la résolution d'équations et de systèmes d'équations, grâce à ses algorithmes spécialisés qui exploitent pleinement les capacités des processeurs Intel. En C comme en Python, il permet d’accélérer les calculs tout en garantissant précision et efficacité. Cet outil puissant et flexible est essentiel pour les applications scientifiques nécessitant des calculs complexes, en assurant des gains de temps significatifs dans le traitement des données.

Bibliographie

Intel

Architecture Support

The browser version you are using is not recommended for this site.Please consider upgrading to the latest version of your browser by clicking one of the following links.

Intel

FFT Code Examples

The browser version you are using is not recommended for this site.Please consider upgrading to the latest version of your browser by clicking one of the following links.

Intel

Development Tools

Search for development software and tools from Intel the way you want.

www.netlib.org

BLAS (Basic Linear Algebra Subprograms)

The BLAS (Basic Linear Algebra Subprograms) are routines that provide standard building blocks for performing basic vector and matrix operations. The Level 1 BLAS perform scalar, vector and vector-vector operations, the Level 2 BLAS perform matrix-vector operations, and...

www.netlib.org

LAPACK — Linear Algebra PACKage

Version 3.12.1 LAPACK on GitHub Browse the LAPACK User Forum Browse the LAPACK User Forum Contact the LAPACK team Get the latest LAPACK News # access

Python documentation

ctypes — A foreign function library for Python

Source code: Lib/ctypes ctypes is a foreign function library for Python. It provides C compatible data types, and allows calling functions in DLLs or shared libraries. It can ...

Intel

Development Tools

Search for development software and tools from Intel the way you want.